

BBH Chatbot Plugin

Giới thiệu

BBH Chatbot Plugin là một ứng dụng React TypeScript được thiết kế để nhúng widget chatbox vào các **website**. Plugin này cung cấp giao diện chat thời gian thực giữa khách hàng (visitor) và doanh nghiệp thông qua nền tảng **WEBSITE**, đồng thời hỗ trợ tích hợp AI chatbot.

Lưu ý: Plugin này **chỉ hoạt động trên nền tảng Website**. Các nền tảng khác như Facebook Messenger, Zalo, WhatsApp,... chỉ xuất hiện dưới dạng **liên kết nhanh (Social Links)** trong giao diện Home để khách hàng có thể chuyển sang các kênh liên lạc khác của doanh nghiệp.

Mục lục

- [Tính năng](#)
- [Công nghệ sử dụng](#)
- [Cấu trúc dự án](#)
- [Cài đặt](#)
- [Cấu hình môi trường](#)
- [Scripts](#)
- [Kiến trúc ứng dụng](#)
- [API Endpoints](#)
- [Components chính](#)
- [Hooks](#)
- [State Management](#)
- [WebSocket](#)
- [Đa ngôn ngữ](#)

Tính năng

Chat Realtime trên Website

- Gửi/nhận tin nhắn văn bản theo thời gian thực qua WebSocket

- Hỗ trợ upload và gửi hình ảnh, video, audio, file đính kèm
- Hiển thị trạng thái typing (đang soạn tin)
- Đếm số tin nhắn chưa đọc
- Quick Chat popup hiển thị tin nhắn mới



AI Chatbot

- Tích hợp trợ lý ảo AI (AI Agent)
- Hỗ trợ câu hỏi thường gặp (FAQ / CTA Messages)
- Quick replies - gợi ý câu trả lời nhanh
- Hiển thị nguồn tham khảo từ LLM (llm_sources)



Giao diện tùy chỉnh

- Tùy chỉnh màu sắc chủ đạo (primary color, text color)
- Custom logo trang
- Tùy chỉnh vị trí hiển thị widget (bottom_right, bottom_left, etc.)
- Hiệu ứng nút trigger (button effect)
- Welcome message - tin nhắn chào mừng
- Custom background



Responsive

- Hỗ trợ hiển thị trên Desktop và Mobile
- Chế độ view_screen cho full-screen (embed trong mobile app)
- Tự động điều chỉnh layout theo kích thước màn hình



Liên kết nhanh (Social Links)

Trên màn hình Home, khách hàng có thể bấm vào các liên kết để chuyển sang kênh liên lạc khác của doanh nghiệp:

-  Facebook / Messenger
-  Zalo
-  WhatsApp
-  Instagram
-  Telegram
-  Website khác
-  Email

- 📞 Điện thoại
- Và nhiều nền tảng khác (TikTok, YouTube, Twitter, LinkedIn, Discord, Line, Viber,...)

🌐 Đa ngôn ngữ

Hỗ trợ 6 ngôn ngữ:

- 🇻🇳 Tiếng Việt (vn)
- 🇺🇸 Tiếng Anh (en)
- 🇨🇳 Tiếng Trung (cn)
- 🇯🇵 Tiếng Nhật (jp)
- 🇰🇷 Tiếng Hàn (kr)
- 🇹🇭 Tiếng Thái (th)

✉️ Template Messages

Hỗ trợ hiển thị các loại tin nhắn template từ server:

- Generic Template (carousel với nhiều item)
- Button Template (tin nhắn với các nút bấm)
- Media Template (video, image với buttons)
- Coupon Template (mã giảm giá)
- Receipt Template (hóa đơn/đơn hàng)
- Customer Feedback Template (đánh giá)

👤 Form đăng ký khách hàng

- Form trước khi chat để thu thập thông tin khách hàng (tên, SĐT, email)
- Tùy chỉnh bật/tắt từng trường
- Validation theo cấu hình

Công nghệ sử dụng

Công nghệ	Phiên bản	Mục đích
React	^18.3.1	UI Framework
TypeScript	^5.6.2	Type Safety

Công nghệ	Phiên bản	Mục đích
Vite	^5.4.5	Build Tool
Redux Toolkit	^2.2.7	State Management
TailwindCSS	^3.4.11	Styling
React Router	^6.26.2	Routing
i18next	^23.15.1	Internationalization
Framer Motion	^12.23.12	Animations
WebSocket	Native	Real-time Communication
bbh-chatbox-widget-js-sdk	^1.0.13	Widget SDK

Cấu trúc dự án

```
chatbot-plugin/
├── public/                # Static assets
├── src/
│   ├── api/              # API configuration & endpoints
│   │   ├── api.ts        # API utilities & fetch functions
│   │   ├── env/          # Environment configurations
│   │   ├── index.ts      # API exports
│   │   └── types.d.ts    # API types
│   ├── assets/           # Images, icons, SVGs
│   │   └── Logo/         # Social platform logos
│   ├── components/       # Reusable components
│   │   ├── ChatComponents/ # Chat-related components
│   │   │   ├── Body/     # Chat body (InitClient, InputChat, MessageBody)
│   │   │   ├── DetailChat/ # Chi tiết màn chat
│   │   │   ├── Header/   # Chat header
│   │   │   ├── MessageComponent/ # Render các loại tin nhắn
│   │   │   ├── Modal/    # Modal components
│   │   │   └── type.d.ts  # Component types
│   │   ├── HomeComponents/ # Home screen components
│   │   │   ├── ChatOption.tsx # Liên kết nhanh (Social Links)
│   │   │   ├── FAQ.tsx       # Câu hỏi thường gặp
│   │   │   ├── SendMessage.tsx # Nút gửi tin nhắn
│   │   │   └── UnreadMessage.tsx # Tin nhắn chưa đọc
│   │   ├── Container/      # Layout containers
│   │   ├── Icon/           # Icon components
│   │   ├── Loading/        # Loading indicators
│   │   ├── WebSocket/      # WebSocket handler
│   │   └── ...              # Other shared components
│   ├── hooks/             # Custom React hooks
│   │   ├── useApp.ts       # Main app hook (init, postMessage)
│   │   └── useChathandlers.ts # Chat action handlers
│   ├── lang/              # Translation files
│   │   ├── cn.ts          # Chinese
│   │   └── en.ts          # English
```

- └─ jp.ts # Japanese
- └─ kr.ts # Korean
- └─ th.ts # Thai
- └─ vn.ts # Vietnamese
- └─ pages/ # Page components
 - └─ ChatApp/ # Main chat application
 - └─ ChatApp.tsx # Chat app component
 - └─ useChatApp.ts # Chat app logic hook
 - └─ useChatAppAction.ts # Chat actions
 - └─ components/ # Sub-components
 - └─ Header.tsx # Header popup
 - └─ Menu.tsx # Tab menu (Home/Message)
 - └─ QuickChat.tsx # Quick chat popup
 - └─ TriggerButton.tsx # Nút mở chatbox
 - └─ WelcomeMessage.tsx # Tin chào mừng
 - └─ FeedbackModal.tsx # Modal đánh giá
 - └─ OrderConfirmation.tsx # Xác nhận đơn
 - └─ ActiveSDK.tsx # Trang kích hoạt SDK
 - └─ type.d.ts # Page types
- └─ screens/ # Screen components
 - └─ AIScreen/ # AI assistant screens
 - └─ ChatScreen/ # Chat screens
 - └─ Chat/ # Màn chat chính
 - └─ Chat.tsx
 - └─ useChatClient.ts
 - └─ Home.tsx # Màn Home (greeting, FAQ, social links)
 - └─ type.d.ts
- └─ stores/ # Redux store
 - └─ appSlice.ts # App state slice (reducers, selectors)
 - └─ index.ts # Store configuration
 - └─ type.d.ts # State types
- └─ utils/ # Utility functions
 - └─ index.ts # Helper functions
 - └─ constants.ts # Constants (LANGUAGE_MAP)
 - └─ type.ts # Common types
- └─ App.tsx # Root component với Routes
- └─ App.css # Global styles

```
|   |─ i18n.ts           # i18n configuration
|   └─ index.tsx        # Entry point
|
|─ .env                 # Development environment
|─ .env.production      # Production environment
|─ package.json         # Dependencies
|─ tailwind.config.js   # TailwindCSS config
|─ tsconfig.json        # TypeScript config
|─ vite.config.ts       # Vite config
└─ vercel.json          # Vercel deployment
```

Cài đặt

Yêu cầu

- Node.js >= 16.x
- pnpm >= 8.x (khuyến nghị)

Cài đặt dependencies

```
# Sử dụng pnpm (khuyến nghị)
pnpm install
```

```
# Hoặc npm
npm install
```

```
# Hoặc yarn
yarn install
```

Cấu hình môi trường

Tạo file `.env` với các biến sau:

```
# API Host – Backend server
VITE_APP_BE_HOST=https://api.example.com

# WebSocket Host – Realtime messaging
VITE_APP_SOCKET_HOST=wss://socket.example.com

# Image CDN
VITE_IMAGE_HOST=https://images.example.com
VITE_CDN=https://cdn.example.com

# Widget Configuration
VITE_ID_WIDGET=your-widget-id
VITE_APP_URL=https://app.example.com
VITE_WIDGET_URL=https://widget.example.com
VITE_CHATBOT_URL=https://chatbot.example.com
VITE_APP_V2_URL=https://app-v2.example.com

# Domain cho trigger button
VITE_DOMAIN_TRIGGER_BTN=example.com

# Environment
VITE_APP_ENV=development # hoặc 'production'
```

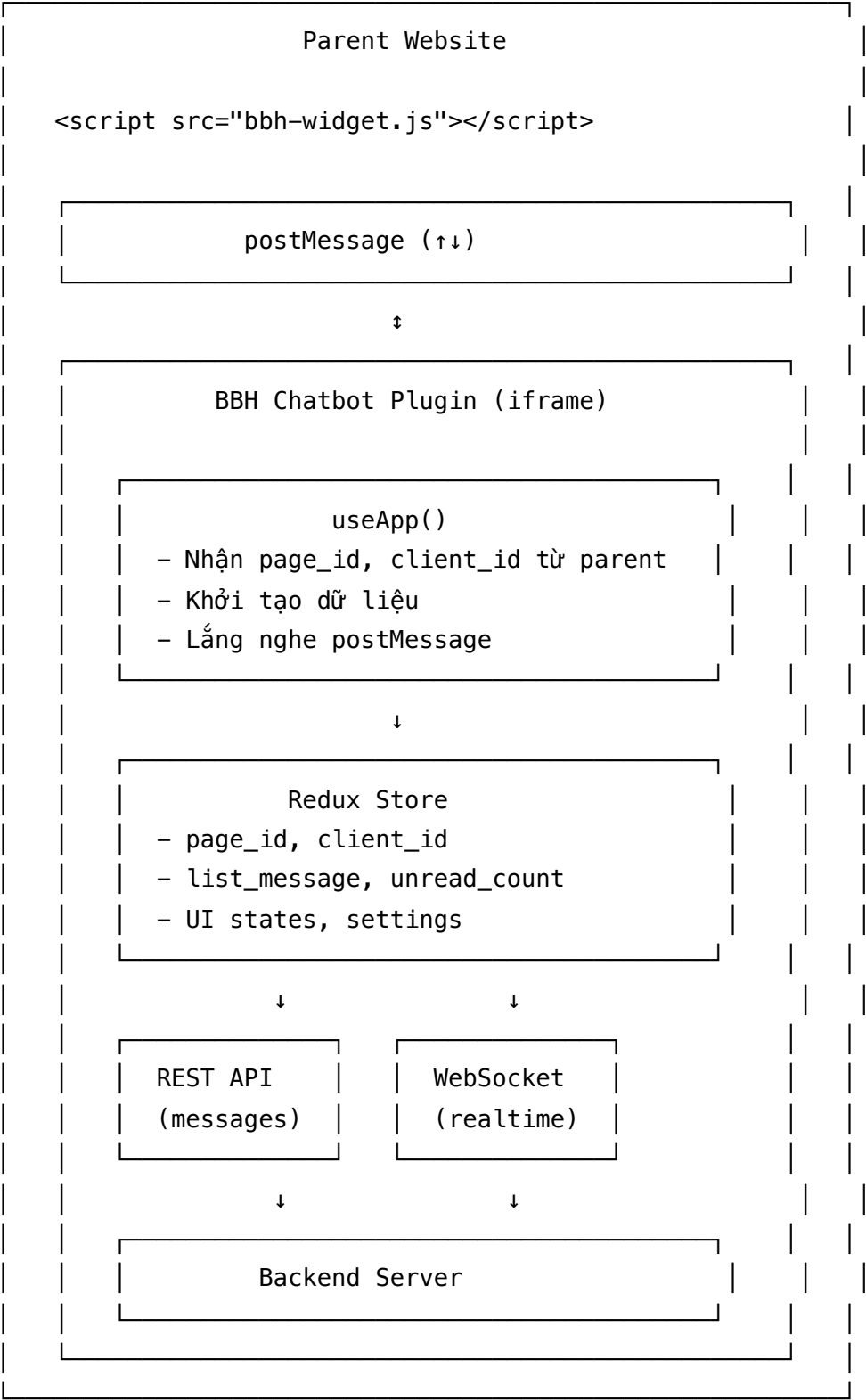
Scripts

Script	Mô tả
pnpm dev	Chạy development server
pnpm build	Build production bundle
pnpm preview	Preview production build
pnpm start:staging	Chạy với staging environment
pnpm start:prod	Chạy với production environment

Kiến trúc ứng dụng

Cách hoạt động

Plugin được embed vào website dưới dạng **iframe**. Giao tiếp giữa website cha (parent) và plugin thông qua **postMessage API**.



Routes

Route	Component	Mô tả
/	ChatApp	Trang chat chính với 2 tab: Home & Message

Route	Component	Mô tả
/ai-assistant	ChatApp	Chế độ AI Assistant (không có tab Home)
/view-screen	ChatApp	Chế độ full-screen (cho mobile app, web app)
/active-sdk	ActiveSDK	Trang kích hoạt widget

Luồng khởi tạo

1. **Parent website** load widget script và khởi tạo với `page_id`
2. **Widget** tạo `iframe` và `embed plugin`
3. **Plugin** nhận `page_id` qua `postMessage` hoặc `URL params`
4. **useApp** hook gọi API lấy thông tin trang (`settings`, `staff`, `AI config`,...)
5. **Nếu có `client_id`** đã lưu → load lịch sử chat
6. **Nếu chưa có `client_id`** → hiển thị form đăng ký (nếu bật)
7. **WebSocket** kết nối để nhận tin nhắn realtime

API Endpoints

REST APIs

```
const API_END_POINTS = {  
  // Đọc danh sách tin nhắn của cuộc hội thoại  
  READ_MESSAGE_API: '/embed/message/read_message',  
  
  // Gửi tin nhắn (text, attachment)  
  SEND_MESSAGE_API: '/embed/message/send_message',  
  
  // Khởi tạo/đăng ký client mới  
  INIT_CLIENT_API: '/embed/conversation/init_identify',  
  
  // Đọc thông tin cấu hình trang (settings, staff, AI,...)  
  READ_PAGE_INFO: '/embed/page/read_page',  
  
  // Đọc thông tin khách hàng  
  READ_CLIENT_INFO: '/embed/conversation/read_client',  
}
```

WebSocket Events

```
// Gửi khi kết nối để xác thực
{
  page_id: string,
  client_id: string,
  event: 'JOIN'
}

// Nhận tin nhắn mới
{
  message?: MessageInfo,
  sender_action?: 'typing_on' | 'typing_off',
  quick_replies?: QuickReply[],
}
```

Components chính

ChatApp

Component container chính của ứng dụng chat:

```
ChatApp
├── Header          # Logo, thông tin trang, nút đóng
├── Home            # Màn Home (greeting, FAQ, social links)
├── ChatScreen      # Màn Chat (tin nhắn, input)
├── Menu            # Tab navigation (Home/Message)
├── QuickChat       # Popup tin nhắn nhanh
├── WelcomeMessage  # Tin nhắn chào mừng
├── TriggerButton   # Nút mở/đóng chatbox
├── Modal           # Preview ảnh
├── FeedbackModal   # Đánh giá
└── OrderConfirmation # Xác nhận đơn hàng
```

Home Screen

Màn hình chào mừng với các thành phần:

- **Greeting:** Lời chào theo tên khách hàng
- **UnreadMessage:** Hiển thị tin nhắn chưa đọc
- **SendMessage:** Nút bắt đầu chat
- **ChatOption:** Liên kết nhanh đến các kênh liên lạc khác (Social Links)
- **FAQ:** Câu hỏi thường gặp (CTA Messages)

ChatScreen / DetailChat

Màn hình chat chính:

- **ChatHeader:** Header với thông tin trang/nhân viên
- **MessageBody:** Render danh sách tin nhắn
- **MessageComponent:** Render từng tin nhắn theo loại
- **InputChat:** Input gửi tin nhắn + upload file
- **InitClient:** Form đăng ký thông tin khách hàng

MessageComponent

Render các loại tin nhắn:

- Text messages
- Image/Video/Audio attachments
- Template messages (Generic, Button, Media, Receipt, Coupon, Feedback)
- Reply messages (tin nhắn trả lời)

ChatOption (Social Links)

Hiển thị danh sách liên kết nhanh để khách hàng chuyển sang kênh khác:

```
// Các loại social link được hỗ trợ
type SocialLinkType =
  | 'LINK_WEB' // Website
  | 'PHONE' // Số điện thoại
  | 'MAIL' // Email
  | 'FACEBOOK' // Facebook page
  | 'MESSENGER' // Facebook Messenger
  | 'INSTAGRAM' // Instagram
  | 'WHATSAPP' // WhatsApp
  | 'ZALO' // Zalo
  | 'TELEGRAM' // Telegram
  | 'TIKTOK' // TikTok
  | 'YOUTUBE' // YouTube
  | 'TWITTER' // Twitter/X
  | 'LINKEDIN' // LinkedIn
  | 'DISCORD' // Discord
  | 'LINE' // Line
  | 'VIBER' // Viber
  | 'THREADS' // Threads
  | 'REDDIT' // Reddit
  | 'PINTEREST' // Pinterest
  | 'GITHUB' // GitHub
  | 'TWITCH' // Twitch
  | 'VK' // VK
  | 'APP_STORE' // App Store
  | 'GOOGLE_PLAY_STORE' // Google Play
```

Hooks

useApp

Hook chính khởi tạo và quản lý ứng dụng:

```
const {
  PAGE_ID, // ID trang
  stored_client_id, // ID khách hàng (từ localStorage)
  is_show, // Trạng thái hiển thị popup
  setShow, // Toggle popup
  handleToggle, // Xử lý toggle (PC mode)
  handleOff, // Đóng popup (Mobile mode)
  type_consultation, // Trạng thái tư vấn
  setTypeConsultation, // Set trạng thái tư vấn
} = useApp()

// Các function nội bộ:
// - fetchData(): Lấy thông tin trang từ API
// - decodeClientData(): Giải mã thông tin client từ params
// - handleMessage(): Xử lý postMessage từ parent website
// - fetchPageSetting(): Lấy cấu hình trang
// - fetchClientData(): Lấy thông tin khách hàng
```

useChatApp

Hook xử lý logic màn chat:

```

const {
  // States
  LIST_MESSAGE,           // Danh sách tin nhắn
  LATEST_MESSAGE,         // Tin nhắn mới nhất
  GLOBAL_UNREAD_MESSAGE_COUNT, // Số tin chưa đọc
  current_tab,            // Tab hiện tại (home/message)

  // Settings
  AI_STATUS,              // Có phải chế độ AI không
  IS_VIEW_SCREEN,         // Có phải chế độ full-screen không
  POSITION,                // Vị trí widget
  CUSTOM_COLOR,           // Màu tùy chỉnh

  // Staff info
  EMPLOYEE_LIST,          // Danh sách nhân viên
  staff_list,             // Thông tin staff

  // Social links
  social_link,            // Danh sách liên kết nhanh
  social_description,     // Mô tả cho social links

  // Functions
  getContainerLayout(),   // CSS layout theo trạng thái
  getMainPopupLayout(),  // CSS popup chính
  getQuickchatLayout(),  // CSS quick chat
  handleCloseModal(),    // Đóng modal
} = useChatApp({ show })

```

useChatHandlers

Hook xử lý các action:


```
const {  
  handleBtn, // Xử lý click nút trigger  
  setHideForMobile, // Đóng popup trên mobile  
} = useChatHandlers({  
  is_show,  
  setShow,  
  handleToggle,  
  handleOff,  
  PAGE_ID,  
  CLIENT_ID,  
  setTypeConsultation,  
})
```

State Management

AppState

```
interface AppState {  
    // ===== Identity =====  
    page_id: string // ID trang (từ server)  
    client_id: string // ID khách hàng  
  
    // ===== Messages =====  
    list_message: MessageInfo[] // Danh sách tin nhắn  
    list_unread_message: MessageInfo[] // Tin chưa đọc  
    latest_message?: MessageInfo // Tin nhắn mới nhất  
    unread_count: number // Số tin chưa đọc  
  
    // ===== UI States =====  
    show_popup: boolean // Popup đang mở/đóng  
    loading_global: boolean // Loading toàn cục  
    is_init: boolean // Đã khởi tạo client (qua form)  
    current_width: number // Chiều rộng màn hình parent  
    current_height: number // Chiều cao màn hình  
    preview_url?: string // URL ảnh đang preview  
  
    // ===== AI =====  
    is_ai?: boolean // Chế độ AI  
    page_info_ai?: {  
        // Thông tin AI agent  
        ai_agent_id?: string  
        is_active_ai_agent?: boolean  
        current_staff_name?: string  
    }  
    typing_status?: boolean // Trạng thái đang soạn tin  
  
    // ===== Customization =====  
    custom_color?: {  
        // Màu tùy chỉnh  
        primary_color?: string  
        text_color?: string  
    }  
    embed_position?: string // Vị trí widget  
    embed_position_detail?: {  
        // Chi tiết vị trí
```

```

    bottom?: number
    right?: number
    left?: number
}
is_custom_background?: boolean // Có custom background
button_effect?: boolean // Hiệu ứng nút trigger

// ===== Staff =====
staff_list?: EmployeeList // Danh sách nhân viên
is_visible_staff?: {
    // Cấu hình hiển thị nhân viên
    is_active?: boolean
    allow_staffs?: EmployeeList
}

// ===== Features =====
quick_chat?: any[] // Quick chat data (từ socket)
form_before_chat?: {
    // Form trước khi chat
    is_active: boolean
    data?: FormData[]
}
faq_question_cta?: FAQ_QUESTION_CTA // FAQ/CTA
is_active_cta_message?: boolean
is_visible_home_page?: boolean // Hiển thị trang Home
open_popup_when_access?: {
    // Tự động mở popup
    option?: string
    delay?: number
}

// ===== Branding =====
org_allow_logo?: boolean // Cho phép custom logo
logo_page_custom?: string // Logo tùy chỉnh
page_avatar?: string // Avatar trang

// ===== User Info =====
user_info?: {
    user_name: string
    user_phone: string
    user_email: string
    client_id: string
}

```

```
    client_name_store?: string // Tên khách hàng  
}
```

Reducers chính

```
// Lưu page_id  
setPageId(state, action: PayloadAction<string>)  
  
// Lưu client_id  
setGlobalClientId(state, action: PayloadAction<string>)  
  
// Cập nhật danh sách tin nhắn  
setListMessage(state, action: PayloadAction<MessageInfo[]>)  
  
// Thêm tin nhắn chưa đọc  
setListUnreadMessage(state, action: PayloadAction<MessageInfo[]>)  
  
// Cập nhật số tin chưa đọc  
setGlobalUnreadCount(state, action: PayloadAction<number>)  
  
// Cập nhật trạng thái typing  
setTypingStatus(state, action: PayloadAction<boolean>)  
  
// Lưu thông tin người dùng  
setUserInfo(state, action: PayloadAction<UserInfo>)  
  
// Reset cuộc hội thoại  
resetConversation(state)
```

WebSocket

Kết nối và xác thực

```
// Khởi tạo WebSocket
const WS = new WebSocket(SOCKET_API)

// Khi kết nối thành công
WS.onopen = () => {
  // Gửi tin nhắn JOIN để xác thực
  WS.send(
    JSON.stringify({
      page_id: page_id,
      client_id: client_id,
      event: 'JOIN',
    })
  )

  // Ping mỗi 25 giây để giữ kết nối
  setInterval(() => WS.send('ping'), 25000)
}
```

Xử lý tin nhắn

```
WS.onmessage = ({ data }) => {  
  // Bỏ qua pong  
  if (data === 'pong') return  
  
  const socket_data = JSON.parse(data)  
  
  // Xử lý typing status  
  if (socket_data.sender_action === 'typing_on') {  
    dispatch(setTypingStatus(true))  
  }  
  if (socket_data.sender_action === 'typing_off') {  
    dispatch(setTypingStatus(false))  
  }  
  
  // Xử lý tin nhắn mới  
  if (socket_data.message) {  
    // Nếu popup đóng hoặc đang ở tab Home  
    if (!IS_SHOW || current_tab !== 'message') {  
      dispatch(setListUnreadMessage([...current, message]))  
      dispatch(setGlobalUnreadCount(count + 1))  
    }  
    dispatch(setLatestMessageGlobal(message))  
  }  
  
  // Xử lý quick replies  
  if (socket_data.quick_replies) {  
    dispatch(setDataQuickChat(socket_data.quick_replies))  
  }  
}
```

Auto-reconnect

```
WS.onclose = () => {  
  // Tự động kết nối lại sau 100ms (trừ khi force close)  
  if (!is_force_close_socket) {  
    setTimeout(() => onSocketFromChatboxServer({...}), 100)  
  }  
}
```

Đa ngôn ngữ

Cấu hình i18n

```
// src/i18n.ts
import i18n from 'i18next'
import LanguageDetector from 'i18next-browser-languagedetector'

i18n.use(LanguageDetector).init({
  resources: {
    vi: { translation: vnTranslation },
    en: { translation: enTranslation },
    cn: { translation: cnTranslation },
    jp: { translation: jpTranslation },
    kr: { translation: krTranslation },
    th: { translation: thTranslation },
  },
  fallbackLng: 'vi',
})
```

Sử dụng trong component

```
import { useTranslation } from 'react-i18next'

function MyComponent() {
  const { t, i18n } = useTranslation()

  return (
    <div>
      <h1>{t('welcome')}</h1>
      <button>{t('send')}</button>
      <p>{t('errorMessage')}</p>
    </div>
  )
}
```

Language mapping

```
// Mapping locale code -> tên dùng trong API
const LANGUAGE_MAP: Record<string, string> = {
  vi: 'vi',
  en: 'en',
  cn: 'zh',
  jp: 'ja',
  kr: 'ko',
  th: 'th',
}
```

Các Types quan trọng

MessageInfo

```
interface MessageInfo {
  _id: string
  fb_page_id: string
  fb_client_id: string
  message_type: 'page' | 'client' | 'system' | 'note' | 'activity'
  message_text?: string
  message_attachments?: AttachmentInfo[]
  time: string
  message_mid?: string
  platform_type?: 'WEBSITE'
  llm_sources?: { link?: string; title?: string }[]
  snap_replay_message?: MessageInfo // Tin nhắn được reply
  sender_action?: 'typing_on' | 'typing_off'
}
```


AttachmentInfo

```
interface AttachmentInfo {
    type?: 'image' | 'video' | 'audio' | 'file' | 'template' | 'fallback'
    url?: string
    title?: string
    payload?: {
        url?: string
        template_type?:
            | 'button'
            | 'generic'
            | 'media'
            | 'receipt'
            | 'coupon'
            | 'customer_feedback'
        elements?: AttachmentPayload[]
        buttons?: ChatbotButton[]
        title?: string
        subtitle?: string
        // ... các field khác tùy template
    }
}
```

ChatbotButton

```
interface ChatbotButton {
    type?:
        | 'postback'
        | 'web_url'
        | 'phone_number'
        | 'bbh_place_order'
        | 'bbh_create_transaction'
        | 'bbh_schedule_appointment'
    title?: string
    payload?: string // Cho postback
    url?: string // Cho web_url
}
```

Deployment

Vercel

```
# Build production
pnpm build
```

```
# Deploy
vercel --prod
```

Manual Deploy

```
# Build
pnpm build:prod

# Output trong thư mục dist/
# Upload dist/ lên server của bạn
```

Troubleshooting

Lỗi WebSocket không kết nối

- Kiểm tra URL WebSocket trong environment (VITE_APP_SOCKET_HOST)
- Đảm bảo server WebSocket đang chạy
- Kiểm tra CORS policy
- Xem console log: "WebSocket Connected!" hoặc "WebSocket Disconnected"

Tin nhắn không hiển thị realtime

- Kiểm tra WebSocket connection trong browser DevTools → Network → WS
- Xác minh page_id và client_id đúng
- Kiểm tra socket event 'JOIN' đã được gửi (xem console)
- Đảm bảo nhận được message trong onmessage handler

PostMessage không hoạt động

- Đảm bảo iframe được embed đúng cách
- Kiểm tra origin của parent website
- Xem console log để debug message events
- Kiểm tra handler `handleMessage` trong `useApp`

Form không hiển thị

- Kiểm tra cấu hình `form_before_chat` từ server
- Đảm bảo `is_active: true`
- Kiểm tra đã có `client_id` trong `localStorage` chưa

AI không phản hồi

- Kiểm tra `is_active_ai_agent` trong settings
- Xác minh `ai_agent_id` hợp lệ
- Kiểm tra console có lỗi gì không

Contributing

1. Fork repository
2. Tạo branch feature mới (`git checkout -b feature/amazing-feature`)
3. Commit changes (`git commit -m 'Add some amazing feature'`)
4. Push lên branch (`git push origin feature/amazing-feature`)
5. Tạo Pull Request

License

© 2024 BBH Chatbot. All rights reserved.